

SOFTWARE TESTING & VALIDATION · FINAL PROJECT

EasyService: Defect Log & Quality Metrics — Parts F & G (v1.0)

Institution: Addis Ababa University — School of Information Technology and Engineering

Course: Software Testing and Validation

Document: Defect Log & Quality Metrics (Parts F & G)

Repository: <https://github.com/Abyman2/Easyservice>

Group Members

#	Full Name	Student Number	Email Address	GitHub Username
1	**Abel Seleshe**	ATE/6743/14	abelseleshi24@gmail.com	[Abyman2](https://github.com/Abyman2)
2	**Baheran Tesfaye**	ATE/5750/14	Bahrantesfaye1@gmail.com	[Bahrann](https://github.com/Bahrann)
3	**Nebiyu Yohannes**	ATE/3973/14	natijhonny@gmail.com	[NebiyuYohannes](https://github.com/NebiyuYohannes)
4	**Wondesen Teshale**	ATE/4671/14	wendesentesha16@gmail.com	[WondesenTeshale](https://github.com/WondesenTeshale)

1. Defect Management Strategy

Defects in EasyService were identified during continuous integration test suite execution and exploratory testing. Every defect is documented with full steps to reproduce, root cause analysis, severity classification, and a mandatory regression test.

2. Master Defect Log

Defect DEF-001: Unverified Customers Able to Book Listings

- **Severity:** High | **Priority:** High | **Status:** Closed
- **Summary:** `BookingTransactionService` failed to validate customer `IdentityStatus` before initiating transaction.
- **Root Cause:** Missing check for `customer.getIdentityStatus() == IdentityStatus.VERIFIED` in initial service draft.
- **Fix:** Added BR-02 verification guard check throwing `IllegalStateException`.
- **Regression Test:** `BookingTransactionServiceTest.createBooking_UnverifiedUser_ThrowsException()`.

Defect DEF-002: Overbooking Last Available Unit Under Concurrent Requests

- **Severity:** High | **Priority:** High | **Status:** Closed
- **Summary:** Inventory availability check allowed request when `quantity > availableQuantity`.
- **Root Cause:** BVA boundary condition used `>=` instead of `>`.
- **Fix:** Corrected comparison guard in `BookingTransactionService.java`.
- **Regression Test:**
`BookingTransactionServiceTest.createBooking_QuantityExceedsCapacity_ThrowsException()`.

Defect DEF-003: Negative Price Accepted During Provider Listing Creation

- **Severity:** Medium | **Priority:** Medium | **Status:** Closed
 - **Summary:** Provider could create listing with price `$-10.00$`.
 - **Root Cause:** Missing validation check in `ListingService.createListing`.
 - **Fix:** Implemented BR-05 check `price.compareTo(BigDecimal.ZERO) <= 0`.
 - **Regression Test:** `ListingServiceTest.createListing_NegativePrice_ThrowsException()`.
-

3. Quality Metrics Summary

3.1 Test Execution Metrics

- **Total Automated Tests:** 58
- **Passed:** 58 (100%)
- **Failed:** 0
- **Skipped:** 0

3.2 Code Coverage Metrics (JaCoCo)

- **Branch Coverage Target:** `$\ge 80.0\%$`
- **Core service-package Branch Coverage Achieved:** **84.7%** (171/202 branches)
- **Whole-bundle Branch Coverage:** **71.1%** (212/298 branches)
- **Instruction Coverage Achieved:** **91.2%**
- **Classes Analyzed:** 35 core backend classes

3.3 Defect Density & Removal Efficiency (DRE)

- **Total Defects Identified (\$D_{total}\$):** 3
- **Fixed & Verified (\$D_{fixed}\$):** 3
- **Escaped Defects (\$D_{escaped}\$):** 0
- **Defect Removal Efficiency (DRE):**

$$DRE = \frac{D_{fixed}}{D_{fixed} + D_{escaped}} \times 100\% = \frac{3}{3 + 0} \times 100\% = 100\%$$